1b. R objects

ECO1400

1. R objects

Everything in R is generally called an "object". These include data and functions. It is important to distinguish between:

- **Type** of an object, for working with data,
- **Class** of an object, for working with functions.

The type versus class distinction originally comes from programming languages such as C++.

2. R data types

R has several basic data types:

- *character*
- *double* (we can think of this as a real number)
- *integer*
- *logical*
- *complex*

The type of a data object is checked by the function `typeof()`.

```
x <- 1
typeof(x)
```

```
## [1] "double"
```

Elements of these data types are combined into data structures.

3. R Data Structures

Table 1: Data structures in R

Dimensionality	Homogenous Contents	Heterogeneous Contents
1 dimension	atomic vector	list
2 dimensions	matrix	data frame
n dimensions	array	

3.1 Atomic Vector and List

The basic data structure of 1 dimension is a "vector". There are two kinds of vectors: atomic vector and a list. Elements of an atomic vector must all be of the same type. Elements of a list can be of different types.

We can construct atomic vectors by the function `c()`, which we can read as "combine".

```
v <- c(2, 3, 4)
v
```

```
## [1] 2 3 4
```

The type of an atomic vector is the type same as the type of its individual elements.

```
typeof(v)
```

```
## [1] "double"
```

We can construct lists by function `list()`:

```
ls <- list(x, v, c("a", "b", "c"), c(TRUE, FALSE))
ls
```

```
## [[1]]
## [1] 1
##
## [[2]]
## [1] 2 3 4
##
## [[3]]
## [1] "a" "b" "c"
##
## [[4]]
## [1]  TRUE FALSE
```

Since a list contains data of different types, its type is a *list*.

```
typeof(ls)
```

```
## [1] "list"
```

The structure of a list (and any other object) in R is checked by the function `str()`.

```
str(ls)
```

```
## List of 4
##  $ : num 1
##  $ : num [1:3] 2 3 4
##  $ : chr [1:3] "a" "b" "c"
##  $ : logi [1:2] TRUE FALSE
```

3.2 Matrix and Data Frame

A data structure of 2 dimensions of elements of the same type is called a matrix. Matrices are filled in by columns, here from a sequence of numbers 1 to 10 created by the syntax `1:10`.

```r
m <- matrix(1:10, nrow = 5, ncol = 2)
m
```

```
##      [,1] [,2]
## [1,]    1    6
## [2,]    2    7
## [3,]    3    8
## [4,]    4    9
## [5,]    5   10
```

The type of a matrix is the same as of its elements.

```r
typeof(m)
```

```
## [1] "integer"
```

A data structure of 2 dimensions of elements of different types is called a data frame. The elements of a data frame must have the same length (i.e. number of entries). A data frame is thus structured like a matrix but with vectors of different types. As an example, we will construct a data frame of three vectors with types *numerical*, *logical*, and *character*, using the function `data.frame()`:

```r
df <- data.frame(a=c(1,2,3,4), b=c(TRUE,FALSE,TRUE,TRUE), c=c("one","two","three","four"))
str(df)
```

```
## 'data.frame':    4 obs. of  3 variables:
##  $ a: num  1 2 3 4
##  $ b: logi  TRUE FALSE TRUE TRUE
##  $ c: chr  "one" "two" "three" "four"
```

4. Indexing

We can reference an element inside an R data structure with square brackets `[ ]`. For object of 1 dimension v= (2,3,4):

```r
v[2]
```

```
## [1] 3
```

For objects of 2 dimensions:

```r
m[3,2]
```

```
## [1] 8
```

Variables in data frames can also be referenced using the operator `$`.

```r
df$a
```

```
## [1] 1 2 3 4
```

5. Object Attributes: Dimension, Names, Class

An object in R has "attributes" such as dimensions, names, and class.

5.1 Dimension

Dimensions are assigned or checked using the function `dim()`.

The dimensions of the 5x2 matrix m are:

```
dim(m)
```

```
## [1] 5 2
```

The dimensions of the data frame df are:

```
dim(df)
```

```
## [1] 4 3
```

5.2 Names

Names are assigned or checked using the function `names()`.

```
names(v) <- c("two", "three", "four") # assign names to elements of the vector v
names(v) # display the names
```

```
## [1] "two"   "three" "four"
```

5.3 Class

A class of an object describes its design. A class is an important attribute of an R object because many R functions only allow as inputs objects of specific classes. For example, the function `weekdays()` only allows as input an object of class *Date* (or the related time class *POSIXt*), as we will see further below.

In many cases a class of an R object coincides with its type.

```
class(m)
```

```
## [1] "matrix" "array"
```

But in other cases a class differs from the type of an object. For example, an important class in time series analysis is the *Date* class on which we will elaborate in the next section.

There are multiple classes that are grouped together as *numeric* classes, the two most common of which are *integer* and *double* (for double precision floating point numbers, i.e. real numbers). The class *numeric* and *integer* or *double* can thus be used interchangeably.

6. R objects of class Date

We can create an object containing calendar dates using the function `as.Date()`. The function requires an object of class *character* as input (here formatted as "yyyy-mm-dd") and returns an object of class *Date* containing the year, month, and day.

```r
dt <- as.Date("2022-01-01")
dt
```

```
## [1] "2022-01-01"
```

We can check that the function `as.Date()` returns an object of class *Date*.

```r
class(dt)
```

```
## [1] "Date"
```

However, the type of the object dt is *double* (numeric) since dt is made of numbers.

```r
typeof(dt)
```

```
## [1] "double"
```

Many R time series functions only allow arguments of class *Date* and their extensions. This is one of the reasons why the class of an object is so important. Without understanding the class attribute we will not be able to use R time series functions.

As an example, consider the function `weekdays()`, which returns a vector of the day names of its argument. The function only works with argument objects of class *Date*. If we try to input as an argument an object of another class, say *double* (*numeric*), we get an error.

Recall our vector of numbers v, which is of class *numeric* (*double*):

```r
class(v)
```

```
## [1] "numeric"
```

```r
v
```

```
##   two three  four
##     2     3     4
```

The function weekdays applied to it will not run.

```r
weekdays(v)
```

```
## Error in `UseMethod()`:
## ! no applicable method for 'weekdays' applied to an object of class "c('double', 'numeric')"
```

Let's create and object of class *Date*, which is a sequence of calendar dates. We will first create the initial date (object `start`) and final date (object `end`) of class *Date*. We will use the function `seq()` that produces an object of the same class as its `from=` and `to=` arguments.

```r
start <- as.Date("2022-01-01")
end   <- as.Date("2022-01-07")
days  <- seq(from=start, to=end, by="day")
days
```

```
## [1] "2022-01-01" "2022-01-02" "2022-01-03" "2022-01-04" "2022-01-05"
## [6] "2022-01-06" "2022-01-07"
```

We can check the resulting class of `days`:

```
class(days)
```

```
## [1] "Date"
```

If we now pass the object days of class *Date* into the function `weekdays()`, which only accepts arguments of class *Date*, the function will run properly:

```
weekdays(days)
```

```
## [1] "Saturday"  "Sunday"    "Monday"    "Tuesday"   "Wednesday" "Thursday"
## [7] "Friday"
```

The help files of functions typically specify which object classes the functions take as inputs and which object classes they return as the result.

For example, you can check the required class of the arguments and the output object class of the function `weekdays()` either in the R online documentation or using the `help()` function:

```
help(weekdays)
```

The output of the `help()` function will be displayed in the bottom-right pane of RStudio.

In contrast to the few basic data types, there are many different classes of objects in R. For example, the class of the output object of the function `help()` is *help_files_with_topic*:

```
class(help())
```

```
## [1] "help_files_with_topic"
```

One R object can have more than class attribute. As an example, take the function `Sys.time()`, which returns the current date and time:

```
now <- Sys.time()
now
```

```
## [1] "2026-01-13 12:34:55 EST"
```

The class of the output object of `Sys.time()` is of both class *POSIXct* and *POSIXt*, encoding date and time functionality.

```
class(now)
```

```
## [1] "POSIXct" "POSIXt"
```

However, the object is of type *double* (*numeric*), since it is internally made of numerical elements.

```
typeof(now)
```

```
## [1] "double"
```